

Helix OTA — Threat Model (1.0.0-MVP)

Field	Value
Revision	2
Created	2026-06-07
Last modified	2026-06-08
Status	active
Status summary	STRIDE-based threat model for the Helix OTA 1.0.0-MVP: native Android 15 A/B (update_engine + AVB/dm-verity + auto-rollback) on device, a custom Go + Gin control plane, a React dashboard, and a PostgreSQL + MinIO/S3 artifact store. Enumerates twelve OTA-relevant threats with attack, impact, the mitigation actually present in the locked MVP design, and residual risk. Distinguishes MVP-shipped controls from the TUF/Uptane hardening deferred to 1.0.1+ per ADR-0002. Revision 2 adds §4.13 — operational / rollout / recall endpoint threats (audit-log integrity + the reads-not-audited / failed-mutations-not-audited design choice; device-only-own-telemetry IDOR; device-group membership authorization; staged-rollout abuse + HALT-wins safety; recall/rollback authorization + the recall-vs-anti-downgrade tension), grounded in the implemented server/internal/api handlers + per-route RBAC in server.go + the append-only audit/rollback store. Several mitigations depend on items the underlying research carried as UNVERIFIED (RK3588 / Orange Pi 5 Max AVB lock state, rollback-index storage backend, Android 15 IBootControl AIDL surface, catalogue-brick fit for TUF primitives, HelixConstitution clause text). These are marked UNVERIFIED inline and
Issues	

Field	Value
Fixed	must close before the related residual-risk claims become firm. N/A (initial revision). (1) On ADR-0002 1.0.1+ adoption, add TUF/Uptane threat rows (rollback/freeze/mix-and-match/mirror-denial/key-compromise-recovery) and downgrade the residual risk in §4.3, §4.4, §4.5; (2) re-verify the AVB/rollback residual claims (§4.3, §4.12) once the board AVB lock state and rollback-index backend are byte-confirmed; (3) confirm security / Security-KMP catalogue-brick coverage of the signing/verify seam (§4.1, §4.2); (4) reconcile cited §11.4.x clause numbers against the authoritative HelixConstitution; (5) §4.13 open items: resolve the recall-vs-anti-downgrade tension (§4.13.5) — whether a server-driven recall to N-1 is intended to bypass the on-device AVB rollback-index, and if so by what operator-gated mechanism — this needs an operator/architecture decision (per §11.4.6 it is carried UNVERIFIED, not guessed); confirm whether append-only is enforced at the DB grant level (no UPDATE/DELETE privilege on audit_logs / rollback_history) or only at the repository-interface level (§4.13.1); decide whether failed mutations and authorization denials warrant a separate security-event log (§4.13.1).
Continuation	

Table of contents

1. Purpose, scope, and method
2. System under analysis (locked architecture)
3. Trust boundaries, assets, and STRIDE coverage
4. Threats
 - 4.1 Forged / malicious artifact upload
 - 4.2 Signature / signing-key compromise
 - 4.3 Rollback / downgrade attack
 - 4.4 Mix-and-match (partial / inconsistent release)
 - 4.5 Endless-data / decompression-bomb
 - 4.6 MITM on poll / download
 - 4.7 Device impersonation
 - 4.8 Dashboard authentication / authorization bypass

- [4.9 Build-pipeline supply-chain compromise](#)
 - [4.10 Telemetry spoofing](#)
 - [4.11 Denial of update](#)
 - [4.12 Slot corruption](#)
 - [4.13 Operational / rollout / recall endpoint threats](#)
 - [4.13.1 Audit-log integrity / tampering + the reads-not-audited / failed-mutations-not-audited design choice](#)
 - [4.13.2 Device-only-own-telemetry authorization \(IDOR on GET / devices/{id}/telemetry\)](#)
 - [4.13.3 Device-group membership authorization](#)
 - [4.13.4 Staged-rollout abuse \(create / evaluate; HALT-wins as a safety control\)](#)
 - [4.13.5 Recall / rollback authorization + anti-downgrade interplay](#)
 - 5. [Residual-risk summary](#)
 - 6. [Compliance notes \(HelixConstitution\)](#)
 - 7. [Open / UNVERIFIED items](#)
 - 8. [Sources](#)
-

1. Purpose, scope, and method

This document is the STRIDE threat model for the **1.0.0-MVP** of Helix OTA. It analyses only what the MVP ships per the locked architecture in [2026-06-07-helix-ota-design.md](#) §4–§9 and the phased trust decision in [../research/adr/adr-0002-supply-chain-trust.md](#). It does **not** model controls deferred to 1.0.1+; where a threat is only fully closed by a deferred control, that fact is stated explicitly and reflected in the residual risk.

Method. Each threat is described by: **STRIDE category**, **attack** (how it is mounted), **impact** (what it costs if it succeeds), **mitigation in our design** (the control actually present in the MVP, traced to a design/ADR section), and **residual risk** (what remains after the MVP control, including reliance on deferred work).

Anti-bluff (§7.1 / §11.4.6 / §11.4.123). No facts, citations, or capabilities are fabricated. Every mitigation traces to a cited section of the master design, ADR-0002, or the AVB/rollback research note. Items the underlying research carried as unconfirmed are marked **UNVERIFIED**; a mitigation that depends on an UNVERIFIED fact does not yield a firm residual claim until that fact closes. Submodules are named only from the verified catalogue ([documentation_standards.md](#) §9); none are invented.

MVP trust floor (the baseline every threat is measured against). Per ADR-0002 §4.1, the MVP ships **plain per-artifact trust**: SHA-256 (and SHA-512 where available) + a detached signature, verified **server-side on upload** and **device-side before apply**, plus AVB and the A/B engine's own payload-signature check. **TUF/Uptane is explicitly deferred to 1.0.1+** (ADR-0002 §4.3; master §1 non-goals). The MVP therefore knowingly retains a **single-signing-key exposure** and does **not** mitigate the rollback / fast-forward / indefinite-freeze / mix-and-match / malicious-mirror / key-compromise-recovery classes that TUF closes (ADR-0002 §3.1, §5.2). This model makes that retained exposure explicit per threat rather than implying coverage the MVP does not have.

2. System under analysis (locked architecture)

From master design §4 (locked, not re-decided here):

- **Dashboard (React):** secure login, artifact upload, rollout control, fleet health. Reuses UI-Components-React, Dashboard-Analytics-React, Auth-Context-React (master §10).
- **Control plane (Go + Gin):** REST /api/v1 over **HTTP/3 (QUIC) primary** → **HTTP/2 fallback**, **Brotli** with gzip fallback (master §3). Sub-units: artifact intake + validation + signing-verify (ART), rollout engine (ROLL), device registry + inventory (DEV), telemetry ingest (TEL), auth/RBAC (AUTH). Reuses auth, security, database, Storage, observability, eventbus, ratelimiter, middleware, http3, recovery, Herald, config, cache (master §10).
- **Data plane:** PostgreSQL (relational), MinIO/S3 (artifact blobs), OpenTelemetry/Prometheus (master §3, §9).
- **Device (Android 15, Orange Pi 5 Max / RK3588):** KMP OTA agent (poll → download → verify → apply → report), update_engine (Virtual A/B + compression), **AVB / dm-verity + boot_control** (master §4). Reuses Auth-KMP, Security-KMP, Storage-KMP, Config-KMP (master §10).
- **Build pipeline:** emits the flashing image + OTA .zip + mandatory hash file; a **build-pipeline private key signs** and the **public key lives in the device trust store** (master §1, §6).

For 1.0.0 the rollout is **all-at-once**; the staged-rollout engine lands in 1.0.1 (master §5, §8). The artifact-validator, rollout-engine, protocol types, the update_engine bridge, and the OS-adapter are separate decoupled modules (master §4, §10 new-submodule boundaries: ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema).

3. Trust boundaries, assets, and STRIDE coverage

Trust boundaries. (B1) Internet [?] control plane (device poll/download + dashboard). (B2) Dashboard operator [?] control plane (admin actions). (B3) Control plane [?] data plane (PostgreSQL, MinIO/S3, OTEL). (B4) Build pipeline [?] artifact store / signing key. (B5) Control plane [?] device fleet. (B6) On-device: Android userspace (agent) [?] bootloader/AVB/boot_control (the safety boundary AOSP owns, not Helix — android-avb-rollback.md §10).

Primary assets. The signing private key; the device trust store (public key); OTA artifacts + their hashes/signatures in MinIO/S3; the release/deployment + device-inventory records in PostgreSQL; operator credentials / RBAC state; device identity tokens; telemetry stream; the on-device slot/rollback-index state.

STRIDE coverage map (threat → primary STRIDE categories):

#	Threat	S	T	R	I	D	E
4.1	Forged / malicious artifact upload		x				x
4.2		x	x				x

#	Threat	S	T	R	I	D	E
	Signature / key compromise						
4.3	Rollback / downgrade		x			x	
4.4	Mix-and-match		x				
4.5	Endless-data / decompression-bomb					x	
4.6	MITM on poll / download	x	x		x		
4.7	Device impersonation	x		x	x		x
4.8	Dashboard auth bypass	x		x			x
4.9	Build-pipeline supply-chain	x	x	x			x
4.10	Telemetry spoofing	x	x	x		x	
4.11	Denial of update					x	
4.12	Slot corruption		x			x	
4.13	Operational / rollout / recall endpoints	x	x	x	x	x	x

(S = Spoofing, T = Tampering, R = Repudiation, I = Information disclosure, D = Denial of service, E = Elevation of privilege.)

Threat 4.13 spans the full STRIDE set because it bundles five distinct endpoint-class sub-threats (audit tampering/repudiation, telemetry IDOR information-disclosure + privilege, group-membership privilege, rollout-abuse DoS/tampering, recall-abuse tampering/DoS); the per-sub-threat STRIDE focus is stated inline in §4.13.x.

4. Threats

4.1 Forged / malicious artifact upload

- **STRIDE:** Tampering, Elevation of privilege.
- **Attack:** An actor with upload access (or who reaches the intake API) submits a crafted or arbitrary file as an OTA artifact — wrong package, trojaned package, structurally malformed archive, or a file whose declared hash does not match its bytes — aiming to get it published and deployed to the fleet.
- **Impact:** If accepted and deployed, fleet-wide installation of attacker-chosen firmware; potential mass device compromise. This directly violates the operator “safe upload” guarantee (master §1).
- **Mitigation in our design:** Mandatory **server-side validation on upload** — structure, hash, signature, version monotonicity, and target compatibility —

before a release can be published (master §5, §6; ADR-0002 §4.1). Validation is an isolated, independently testable unit (`ota-artifact-validator`; master §4, §10), OS-aware via plugins with no transport coupling. The signing/verify seam is routed through the verified `security / Security-KMP` bricks (ADR-0002 §4.2). The device **re-verifies before apply** (master §6), so an artifact that bypasses server intake still fails on-device, and the A/B engine performs its **own payload-signature check** (ADR-0002 §4.1). Safety-critical signing-verify path targets $\geq 90\%$ coverage under the four-layer + mutation regime (master §13).

- **Residual risk: Low-to-moderate.** Forgery without the signing key is rejected at two independent gates plus the engine check. The dominant residual is the **single signing key**: an attacker who *holds the key* (see §4.2) defeats this entirely, because plain signing “covers the wrong/arbitrary file class **only if** the signing key is intact” (ADR-0002 §1, “Why plain signing is not enough”). TUF target-metadata + thresholds that would harden this are **deferred to 1.0.1+** (ADR-0002 §4.3). **UNVERIFIED**: whether `security / Security-KMP` actually host the verify primitives without bespoke crypto (ADR-0002 §8 item 9).

4.2 Signature / signing-key compromise

- **STRIDE**: Spoofing, Tampering, Elevation of privilege.
- **Attack**: The attacker steals or coerces use of the **build-pipeline private key** (or any online copy of it in the control plane), then signs arbitrary firmware that passes every signature check end-to-end.
- **Impact**: Total break of artifact trust — the attacker can ship arbitrary, validly-signed firmware to the entire fleet. ADR-0002 §2 names the online Go control plane as “a single, attractive signing-key target.”
- **Mitigation in our design**: MVP keeps the signing key as the **build-pipeline** key, not a routinely-online control-plane key; the **public** key (not the private key) lives in the device trust store (master §1, §6). Defense-in-depth that limits blast radius even with a compromised payload path: **AVB + dm-verity** + the A/B **payload check** + server-side upload verification (ADR-0002 §5.2 interim depth). Key custody is routed through `security / Security-KMP` (ADR-0002 §4.2).
- **Residual risk: High and explicitly accepted for MVP.** ADR-0002 §5.2 states plainly: “MVP retains a single-signing-key exposure (no rollback/freeze/mix-and-match/mirror-denial mitigation until 1.0.1+).” There is **no threshold signing, no offline-key custody, and no key-compromise recovery path in the MVP** — those are TUF/Uptane properties scheduled for 1.0.1+ (ADR-0002 §3.2 “key-compromise resilience via thresholds + offline keys”; §4.3 steps 3–4). A stolen key is a fleet-wide compromise until rotation, and even rotation has no metadata-driven revocation in the MVP. This is the single most important residual risk in the MVP and is the explicit motivation for ADR-0002’s phased adoption.

4.3 Rollback / downgrade attack

- **STRIDE**: Tampering, Denial of service.
- **Attack**: The attacker replays an older, **validly-signed** release (a prior-version OTA .zip that still carries a legitimate signature) to push devices back to firmware with known vulnerabilities — a downgrade/rollback that plain signing alone does not detect, since the old artifact’s signature is genuine.

- **Impact:** Fleet returned to vulnerable firmware; subsequent exploitation of the re-introduced vulnerabilities.
- **Mitigation in our design:** Two layers. (1) **Server-side version monotonicity** is part of mandatory upload validation (master §5) and an anti-downgrade/bootloader version check is part of the security model (master §6). (2) On-device **AVB rollback-index / anti-downgrade**: each image carries a rollback index, the device keeps a stored rollback index in tamper-evident storage, and boot is permitted only if `image.rollback_index >= stored_rollback_index`, with the load-bearing ordering rule that the slot is marked **SUCCESSFUL** **before** the stored index is bumped (`android-avb-rollback.md` §2, §8). This rollback-index protection is **bootloader-enforced, not OS-enforced**, so the Helix agent cannot weaken it and an attacker in userspace cannot bypass it.
- **Residual risk: Moderate.** The bootloader rollback index blocks downgrade *across a rollback-index increment* but does **not** block replay of an older release that shares the same rollback index (rollback indexes are bumped at security milestones, not every build), and the MVP has no metadata layer enforcing per-release freshness/expiry. ADR-0002 §3.1 lists rollback and “fast-forward” among the classes plain signing does **not** mitigate; full protection (TUF timestamp/snapshot freshness + monotonic version metadata) is **deferred to 1.0.1+** (ADR-0002 §4.3, §5.1). **UNVERIFIED:** whether the RK3588 / Orange Pi 5 Max build ships a conformant locked AVB and the rollback-index storage backend (RPMB vs persistent partition vs TEE) — if the board is not AVB-locked, layer (2) does not hold (`android-avb-rollback.md` §Issues, §12; ADR-0002 §8 item 8).

4.4 Mix-and-match (partial / inconsistent release)

- **STRIDE:** Tampering.
- **Attack:** The attacker serves a combination of artifacts/metadata that were never released together — e.g. a valid payload paired with a different release’s valid hash file, or partial per-partition images — so each piece is individually authentic but the *set* is inconsistent.
- **Impact:** Devices apply an incoherent firmware combination, risking malfunction or a downgrade of one component while others advance; defeats the “consistent release” assumption.
- **Mitigation in our design:** The MVP treats every artifact as an **opaque target identified by path + length + SHA-256** with its own signature (ADR-0002 §4.2), and validation checks **target compatibility** and version monotonicity at upload (master §5). The native A/B apply is **atomic to the inactive slot** and the engine verifies its **own payload** as a single unit before commit (master §4; ADR-0002 §4.1), which constrains the device to apply one coherent payload rather than an assembled mixture. AVB authenticates the top-level `vbmeta` which transitively authenticates each protected partition’s descriptors (`android-avb-rollback.md` §2, §3), so partition-level substitution within a booted image is detected.
- **Residual risk: Moderate.** The MVP has **no release-level metadata that binds the full set of targets together** (the snapshot/targets binding that TUF provides). ADR-0002 §3.1 explicitly lists **mix-and-match** among the classes plain signing does **not** close; full mitigation is **deferred to 1.0.1+** (ADR-0002 §3.2, §5.1). Within a single signed payload the engine + AVB constrain coherence; across separately-served artifacts/metadata the binding is weak until TUF lands.

4.5 Endless-data / decompression-bomb

- **STRIDE:** Denial of service.
- **Attack:** A malicious or man-in-the-middle response feeds the device (or the server intake) an unbounded/oversized stream, or a small artifact that decompresses to an enormous size — exhausting disk, memory, or download budget. The transport uses **Brotli** content compression (master §3), which is an attack surface for decompression bombs.
- **Impact:** Device storage/memory exhaustion or wedged download; server-side resource exhaustion at intake; failed or stalled updates across the fleet (a denial-of-update vector, cf. §4.11).
- **Mitigation in our design: Length is part of the target identity** — every artifact is identified by **path + length + SHA-256** (ADR-0002 §4.2), so the expected byte length is known in advance and an over-long stream is detectable. The MVP uses **local-verified-file apply**: the artifact is fully downloaded and **verified locally before apply**, chosen over streaming an unverified payload straight to `applyPayload` (ADR-0002 §4.1) — verification gates apply, so an oversized/garbage stream is rejected before it reaches `update_engine`. Server intake validates structure and hash before publish (master §5). `ratelimiter` and middleware bricks are available on the control-plane path (master §10).
- **Residual risk: Moderate.** “Endless-data” is named by ADR-0002 §3.2 among the classes a TUF metadata layer closes (TUF target length is authenticated by signed metadata, not merely declared); in the MVP the length comes from the artifact record rather than from signed, freshness-checked metadata, so a compromised metadata path (cf. §4.2) could mis-state it. The **download-buffer / decompression bounds in the agent and the intake size caps are implementation details not fixed in the locked design** and must be specified in the component specs — **UNVERIFIED** at this revision.

4.6 MITM on poll / download

- **STRIDE:** Spoofing, Tampering, Information disclosure.
- **Attack:** A network attacker intercepts the device^[?]control-plane poll/report or the artifact download, attempting to inject a malicious response, strip transport security, or read device/fleet data in transit.
- **Impact:** Injected fake “update available” pointing at a malicious artifact, suppressed updates, or disclosure of device identifiers/telemetry.
- **Mitigation in our design: TLS 1.3** on all device^[?]server traffic; **HTTP/3 (QUIC)** primary with **HTTP/2 fallback** (master §3, §6) — QUIC carries TLS 1.3 by construction. Even if the transport were broken, the **artifact is re-verified on-device before apply** by hash + signature, plus the A/B payload check and AVB (master §6; ADR-0002 §4.1), so an injected artifact without a valid signature is rejected at the device gate. **Mutual-TLS is recorded as evaluated** for device identity (master §6).
- **Residual risk: Low for artifact integrity** (signature + hash verification is transport-independent), **moderate for confidentiality and for “fake availability” denial**: the MVP commits to TLS 1.3 but **does not mandate certificate pinning or mTLS** (mTLS is only “evaluated”, master §6), so a device that trusts a rogue CA could be fed forged availability/metadata responses (a denial-of-update vector even though it cannot forge the artifact itself). The MVP also has no signed, freshness-checked metadata to authenticate the *availability* response itself (a TUF property deferred to 1.0.1+, ADR-0002 §3.2 “malicious-mirror denial”).

4.7 Device impersonation

- **STRIDE:** Spoofing, Repudiation, Information disclosure, Elevation of privilege.
- **Attack:** An actor presents a forged or stolen device identity to the control plane to enroll a fake device, pull releases targeted at another device, or poison inventory/telemetry as if it were a legitimate fleet member.
- **Impact:** Skewed rollout cohorts and fleet-health data; unauthorized access to releases; inventory poisoning; repudiation of which device did what.
- **Mitigation in our design:** **Device identity is a token bound to a hardware id via the Android KeyStore** (master §6), and **mutual-TLS is recorded as evaluated** (master §6). The device identity/auth path reuses auth (server) and Auth-KMP / Security-KMP (device) (master §10). Device registry + inventory is a distinct unit (DEV; master §4). `api_keys` and `devices` are first-class data-model entities (master §7), and **every admin action is audited** (master §6) supporting non-repudiation on the operator side.
- **Residual risk: Moderate.** KeyStore-bound tokens raise the bar, but the MVP **does not commit to mTLS** (only evaluated) and the per-device “what should this device install” Director-style targeting is **deferred** — ADR-0002 §4.2 only *reserves* the per-device decision for a future Director repo. Token theft from a rooted/compromised device, and the absence of per-device signed targeting in the MVP, remain open. **UNVERIFIED:** the strength of KeyStore binding on the specific RK3588 / Orange Pi 5 Max build (board TEE specifics are carried UNVERIFIED in `android-avb-rollback.md` §12).

4.8 Dashboard authentication / authorization bypass

- **STRIDE:** Spoofing, Repudiation, Elevation of privilege.
- **Attack:** An attacker bypasses dashboard login, escalates a low-privilege operator to release/deploy rights, fixates/replays a session, or reaches a privileged `/api/v1` endpoint directly without going through the UI.
- **Impact:** Unauthorized upload/publish/deploy (which then chains into §4.1), tampering with rollout or device records, and loss of operator accountability.
- **Mitigation in our design:** **OAuth2 / JWT with RBAC** in the AUTH unit (master §4, §6), reusing the auth brick and (dashboard) Auth-Context-React (master §10). RBAC distinguishes operator roles; `users`, `api_keys`, and `audit_logs` are first-class data-model entities and **every admin action is logged** (master §6, §7) for non-repudiation. `ratelimiter`, `middleware`, and `recovery` bricks harden the API edge (master §10). The REST surface is the single mandated entry (`/api/v1`) with gRPC internal-only (master §3), reducing the privileged attack surface.
- **Residual risk: Moderate, implementation-dependent.** The design names RBAC, JWT, and audit logging but the **concrete role matrix, token lifetime/refresh, and per-endpoint authorization checks are component-spec details not fixed in the locked design** — UNVERIFIED at this revision and the main place real bypass bugs would live. Audit logging is stated for “admin actions”; whether it covers all state-changing API calls is unspecified. No MFA is committed in the MVP.

4.9 Build-pipeline supply-chain compromise

- **STRIDE:** Spoofing, Tampering, Repudiation, Elevation of privilege.

- **Attack:** The attacker compromises the build pipeline upstream of signing — poisoned dependency, malicious build step, or tampered build host — so that a **trojaned artifact is signed by the legitimate key** and emitted as a normal release with a valid hash file.
- **Impact:** Same end state as §4.2 (validly-signed malicious firmware fleet-wide), but reached by subverting the build rather than stealing the key — and harder to detect because every downstream check passes.
- **Mitigation in our design:** ADR-0002 §2 names “the online Go control plane is a single, attractive signing-key target” and frames compromise-resilience as a core driver. MVP controls that *limit* (not prevent) blast radius: the **detached-signature + hash artifact integrity** is verified server-side and device-side (master §6; ADR-0002 §4.1); the signing seam is built **MVP-forward behind a go-securesystemslib-compatible signature. Signer abstraction** so a future TUF role signer (offline keys, thresholds) drops in without rework (ADR-0002 §4.2); AVB + dm-verity + A/B payload check are independent on-device layers (master §6).
- **Residual risk: High and explicitly accepted for MVP.** The MVP has **no provenance / attestation / reproducible-build / threshold-signing control** that would catch a *validly signed but trojaned* artifact — these are precisely the TUF/Uptane “compromise resilience” properties **deferred to 1.0.1+** (ADR-0002 §3.2, §4.3 step 5 dual-repo Director+Image split, §5.2). The MVP-forward signer abstraction reduces *future* rework but provides **no MVP-time mitigation** against build subversion. This is a deliberate, documented gap in the phased decision, not an oversight.

4.10 Telemetry spoofing

- **STRIDE:** Spoofing, Tampering, Repudiation, Denial of service.
- **Attack:** An attacker submits fabricated telemetry events (download_started/installing/installed/verifying/success/failure + error codes/health; master §9) — either inflating success or fabricating failures — to mislead fleet health and, in later phases, manipulate the rollout halt/advance logic that consumes these metrics (master §8, §9).
- **Impact:** Wrong fleet-health picture; masked real failures or false alarms via Herald alerting (master §9); in 1.0.1+ this could **drive the staged-rollout gate the wrong way** (force-halt a good rollout, or suppress halt on a bad one) — master §8 notes metrics drive the halt logic.
- **Mitigation in our design:** Telemetry is ingested over the **same TLS 1.3 device↔server channel** and from **devices identified by KeyStore-bound tokens** (master §6), so anonymous spoofing requires a valid device identity (cf. §4.7). The telemetry schema is a distinct, shared, codec-defined contract (ota-telemetry-schema; master §10), and ingest is an isolated unit (TEL) feeding OpenTelemetry/Prometheus via the observability brick (master §4, §9). Rollout halt logic uses **success/error thresholds over a cohort** rather than single events (master §8), diluting individual spoofed events.
- **Residual risk: Moderate.** Telemetry events are **not individually signed** in the MVP, so a compromised/impersonated device (cf. §4.7) can submit plausible false events within its own identity. For 1.0.0 the rollout is **all-at-once** (master §5), so the telemetry→rollout-gate manipulation impact is **deferred until staged rollout lands in 1.0.1** — at which point per-cohort thresholds plus device-identity binding must be re-evaluated as a control. Threshold-based aggregation limits, but does not eliminate, coordinated multi-device telemetry poisoning.

4.11 Denial of update

- **STRIDE:** Denial of service.
- **Attack:** The attacker prevents devices from receiving/applying legitimate updates — flooding the poll/download endpoints, exhausting artifact-store bandwidth, withholding responses at a malicious mirror, or wedging downloads (cf. §4.5, §4.6) — so a needed security fix never reaches the fleet.
- **Impact:** Fleet stuck on vulnerable/buggy firmware; the operator loses the ability to ship a fix or a recall. (This is the inverse of the “safe upload / granular rollout” guarantees — availability of the update channel itself.)
- **Mitigation in our design: Poll interval is 15 min + jitter, configurable** (master §2 D7, §5) — jitter spreads load and avoids thundering-herd self-DoS, and the configurable interval lets operators back off under stress. `ratelimiter`, `middleware`, and `recovery bricks` protect the control-plane edge; cache is available; artifacts are served from MinIO/S3 over the `Storage brick` (master §10). **Failed/withheld updates degrade safely:** native A/B means a device that cannot complete an update simply **keeps running its current good slot** — a denial-of-update never bricks a device (master §1 zero-corruption; `android-avb-rollback.md` §9). Scalability “single board → millions” is an operator hard guarantee (master §1).
- **Residual risk: Moderate.** Availability hardening in the MVP is generic (rate limiting, jitter, CDN-able object store) rather than OTA-specific; there is **no signed, freshness-checked metadata to detect a malicious-mirror freeze attack** (serving stale “no update” indefinitely) — ADR-0002 §3.1, §3.2 name **indefinite-freeze** and **malicious-mirror denial** among classes only TUF closes, **deferred to 1.0.1+**. The MVP can be silently frozen by a mirror/MITM that withholds updates without the device detecting staleness. Concrete rate-limit thresholds and capacity sizing are component-spec / deployment details — **UNVERIFIED** at this revision.

4.12 Slot corruption

- **STRIDE:** Tampering, Denial of service.
- **Attack:** The inactive slot is corrupted — by a malicious/partial payload, a bit-flip during write, storage faults, or an attacker tampering with on-device partition data — such that booting it would yield a broken or compromised system.
- **Impact:** Without protection, a corrupt slot could brick the device or boot a tampered system — a direct hit on the operator “zero system corruption” hard guarantee (master §1).
- **Mitigation in our design:** This is the strongest-covered threat because the MVP **delegates the safety boundary to AOSP + the bootloader** and the Helix agent only drives documented APIs (`android-avb-rollback.md` §10; master §6). The chain: (1) **atomic A/B write to the inactive slot** — the running slot is never touched (master §4); (2) **update_verifier** on first boot into the new slot reads the **care map** and forces `dm-verity` to verify every written block **before the slot is committed** (`android-avb-rollback.md` §2, §6); (3) **dm-verity** per-block SHA-256 hash-tree verification with a single AVB-authenticated root hash, returning **EIO** on mismatch, with **FEC** tolerating isolated bit-rot (`android-avb-rollback.md` §2, §4); (4) **automatic rollback:** a freshly-activated slot starts not-successful with a positive `slot-retry-count`; if it is never marked **SUCCESSFUL** the bootloader marks it unbootable and **falls back to the prior known-good slot** — this is **bootloader-enforced, not OS-enforced** (`android-`

avb-rollback.md §2, §7). Helix **MUST NOT** flip slot flags, write rollback indexes, regenerate/strip vbmeta, disable verify, or call markBootSuccessful itself (android-avb-rollback.md §10). The device path is validated by an emulated A/B apply plus a real Orange Pi 5 Max plan including a **corrupt-slot** → **confirm A/B fallback** test (master §13).

- **Residual risk: Low — conditional on board conformance.** When the chain is intact, a corrupt slot cannot brick the device or boot tampered code: it fails verification and the bootloader falls back. The residual is the **UNVERIFIED board reality**: whether the RK3588 / Orange Pi 5 Max build actually ships a conformant boot_control HAL + **locked AVB**, the exact slot-retry-count default, the rollback-index storage backend, and the Android 15 IBootControl AIDL surface are all carried UNVERIFIED (android-avb-rollback.md §Issues, §12; ADR-0002 §8 item 8). **If AVB is not locked on the shipped board, this entire residual claim does not hold** and must be re-rated. Closing the board-conformance items (Continuation §13 of the AVB note) is a prerequisite for treating this residual as firm.

4.13 Operational / rollout / recall endpoint threats

This subsection models the operational, staged-rollout, and recall endpoints implemented this session under server/internal/api, traced to the concrete per-route RBAC wired in server.go’s protected group (every protected route runs authMiddleware() → requireRole(...) → handler → auditMiddleware(), in that order). Unlike §4.1–§4.12 (which trace to the locked design / ADRs), the mitigations here are read directly from the shipped Go handlers; where a claim depends on something not present in the read code (e.g. DB-level grants, the actual N-1 re-deploy path), it is marked **UNVERIFIED** per §11.4.6.

Verified RBAC, per endpoint (from server.go lines 125–167):

Method + route	requireRole(...)	Handler
GET /audit	admin only	handleListAudit
GET /devices/:deviceId/telemetry	viewer, operator, admin, device	handleDeviceTelemetry
GET /telemetry/overview	viewer, operator, admin	handleTelemetryOverview
POST /groups	operator, admin	handleCreateGroup
GET /groups, GET /groups/:groupId	viewer, operator, admin	handleListGroup / handleGetGroup
PATCH /groups/:groupId	operator, admin	handleUpdateGroup
DELETE /groups/:groupId	admin only	handleDeleteGroup
GET /groups/:groupId/members	viewer, operator, admin	handleListGroupMembers
POST /groups/:groupId/members	operator, admin	handleAddGroupMember
DELETE /groups/:groupId/members/:deviceId	operator, admin	handleRemoveGroupMember
POST /deployments/:deploymentId/rollout	operator, admin	handleCreateRollout

Method + route	requireRole(...)	Handler
GET / deployments/:deploymentId/ rollout	viewer, operator, admin	handleGetRollout
POST / deployments/:deploymentId/ rollout/evaluate	operator, admin	handleEvaluateRollout
POST / deployments/:deploymentId/ recall	operator, admin	handleRecall
GET / deployments/:deploymentId/ rollbacks	viewer, operator, admin	handleListRollbacks

requireRole (middleware.go) admits a request if the JWT carries **any** allowed role; isPrivileged(claims) is true for admin **or** operator **or** viewer (handlers_device.go), and the device role is never privileged. These are load-bearing for §4.13.2.

4.13.1 Audit-log integrity / tampering + the reads-not-audited / failed-mutations-not-audited design choice

- **STRIDE:** Repudiation, Tampering (and the *gap* is an Information-disclosure / accountability risk).
- **Attack:** An operator/admin performs a sensitive action (publish, deploy, rollout-create, recall, group change) and later denies it; or an attacker who reaches the store attempts to alter/delete audit rows to erase their tracks; or an attacker exploits the *known design choice* that **reads and failed mutations are not audited** to probe the system (enumerate resources via GETs, or hammer mutations that fail authorization/validation) without leaving an audit trail.
- **Impact:** Loss of operator accountability / non-repudiation; an erased or incomplete audit trail undermining incident forensics; reconnaissance (read access, brute-force, authorization probing) that is invisible in the audit log.
- **Mitigation in our design (as implemented):**
 - **Append-only store.** The store.Repository interface exposes only AppendAudit + ListAudit for audit (and AppendRollback + ListRollbacks for rollback) — **there is no Update or Delete method for either log.** The Postgres impl is INSERT-only (AppendAudit is a single INSERT INTO helix_ota.audit_logs ...; postgres.go) with a monotonic seq column driving ORDER BY seq; the in-memory impl is a []AuditEntry slice documented “append-only”. So tampering via the application’s own data path is not possible — there is no code path that mutates or removes an audit row.
 - **Correct middleware ordering.** auditMiddleware() runs **after** authMiddleware() + requireRole(...) and **after** the handler (c.Next() first), then logs **only** when the method is mutating (POST/PUT/PATCH/DELETE) **and** the response status is 2xx. Because it runs after RBAC, an **RBAC-rejected request is never audited** (no log spam from denied probes), and the **verified subject** (claims.Subject) is available to attribute the action.
 - **No id leakage into the action verb.** deriveAuditAction uses the gin route template (c.FullPath()), not the raw path, and drops /* path-

param placeholders, so resource ids never end up in the free-text action string; the specific id is captured separately in the structured `resource_id` field via `auditResourceID`. This keeps the action vocabulary a stable SCREAMING_SNAKE_CASE set and avoids accidental disclosure of ids in an aggregate verb.

- **Read restricted.** GET /audit is **admin-only** (`requireRole(RoleAdmin)`), so the trail itself is not readable by operator/viewer/device.
- **Best-effort, fail-open write.** A failing audit sink does **not** fail the user's already-successful request (`_ = s.repo.AppendAudit(...)`). This is a deliberate availability-over-completeness choice and is the integrity *gap* below.
- **Residual risk: Moderate.**
 - **The “reads not audited” choice is intentional and documented** in the handler (`auditMiddleware` doc comment): reads (GET) and failed mutations are out of audit scope. Consequence: enumeration via GETs, and authorization-probing / brute-force via *failed* mutations, leave **no audit trail**. For a system whose primary audit purpose is operator accountability for state changes this is reasonable, but it means the audit log is **not** a security-event log and cannot be relied on for intrusion detection. Whether a separate security-event log (auth failures, RBAC denials, repeated 4xx) is warranted is an open decision (Continuation item 5).
 - **The “failed mutations not audited” choice** means a mutation that passes RBAC but fails in the handler (validation/conflict/500) is invisible — an attacker cannot see *attempted* privileged actions that errored out. Combined with the best-effort write, a successful action whose audit write *fails* is also silently unlogged.
 - **Append-only is enforced at the repository-interface + SQL-statement level, not (in the read code) at the DB-grant level. UNVERIFIED:** whether the deployment revokes UPDATE/DELETE on `helix_ota.audit_logs / rollback_history` from the application DB role. If the app role retains UPDATE/DELETE, a SQL-injection or compromised-credential path could still tamper rows despite the Go interface omitting those methods. This should close at the migration/grant level. (Cross-ref §4.8 dashboard auth — a JWT-forging or RBAC-bypass attacker who reaches admin can read the trail but still cannot delete it through the app.)

4.13.2 Device-only-own-telemetry authorization (IDOR on GET /devices/{id}/telemetry)

- **STRIDE:** Information disclosure, Elevation of privilege.
- **Attack:** A device (holding a device-role token bound to its own id) requests GET /devices/{otherDeviceId}/telemetry to read another device's telemetry history — a classic **IDOR** (insecure direct object reference): the resource id is in the path and the route admits the device role, so RBAC alone (which only checks *role*, not *which* object) would let any device read any device's telemetry.
- **Impact:** Cross-device disclosure of telemetry (install/verify/success/failure events, error codes, versions) — fleet-member privacy leak and reconnaissance of other devices' update state.
- **Mitigation in our design (as implemented):** `handleDeviceTelemetry` performs an explicit **object-level subject check after the role check**: it loads `claims` and, if the caller is **not** privileged (`!isPrivileged(claims)` — i.e. it

is a bare device token) **and** the token subject does not equal the path `deviceId`, it returns **403 Forbidden** (“a device may read only its own telemetry”). Privileged callers (viewer/operator/admin) may read any device’s telemetry by design (fleet operators need cross-device visibility). This is the correct IDOR mitigation pattern: route-level RBAC gates the *role*, the handler gates the *object*. The identical pattern is used for `GET /devices/:deviceId/status` (`handlers_device.go:92`), so the control is consistent across the two device-scoped reads.

- **Residual risk: Low — for this endpoint, conditional on token integrity.** The IDOR is closed for the device role by the subject check. The residual is upstream: the check trusts `claims.Subject`, so it is only as strong as the device-identity token (cf. §4.7 device impersonation — a stolen/forged device token lets the holder read *that device’s* telemetry, and the MVP does not commit to mTLS). `GET /telemetry/overview` returns only **aggregate fleet counts by event type** (no per-device rows) and is **not** offered to the device role (viewer+ only), so it is not an IDOR surface. **UNVERIFIED:** that `claims.Subject` for a device token is exactly the registered `deviceId` (token minting binds `subject=deviceId` in `handlers_device.go`, but the production device-identity binding strength is the §4.7 UNVERIFIED KeyStore item).

4.13.3 Device-group membership authorization

- **STRIDE:** Elevation of privilege, Tampering.
- **Attack:** A low-privilege actor attempts to create/rename/delete device groups or add/remove group members — e.g. to construct a rollout cohort that includes or excludes specific devices, or to delete a group out from under an in-flight rollout.
- **Impact:** Manipulation of the *targeting substrate* for rollouts (groups define cohorts); unauthorized reshaping of which devices receive an update, or denial by deleting/emptying a group. Group membership is a security-relevant input to the rollout/deploy decision.
- **Mitigation in our design (as implemented):** Group writes are operator/admin only (`POST /groups`, `PATCH /groups/:groupId`, `POST/DELETE .../members` all `requireRole(RoleOperator, RoleAdmin)`); **group deletion is admin-only** (`DELETE /groups/:groupId` → `requireRole(RoleAdmin)`) — a deliberately tighter gate than member edits, reflecting that destroying a group is higher-blast-radius than adjusting membership. Reads (`GET /groups`, `GET /groups/:groupId`, `GET .../members`) are viewer+. The device role has **no** access to any group endpoint (not in any group route’s allow-list), so a device cannot enumerate or alter cohort structure. Every successful group mutation is **audited** (§4.13.1): `auditMiddleware` derives `GROUP_CREATE/GROUP_UPDATE/GROUP_DELETE` and, for the members sub-route, rewrites the resource to `group_member` with `GROUP_MEMBER_CREATE/DELETE` (`deriveAuditAction` special-cases the trailing members segment), so add/remove-member actions are attributable to the operator subject.
- **Residual risk: Moderate.** Authorization is **role-coarse, not group-scoped**: any operator can edit **any** group (there is no per-group ownership / ACL — the handlers take only `groupId` and the role, never an ownership check). In a multi-tenant or separation-of-duties deployment this is over-broad; for the single-fleet MVP it is acceptable. Member-add validates only that `device_id` is non-empty and that the group exists (`AddGroupMember` returns `ErrNotFound` for a missing group) — **UNVERIFIED** whether the added `device_id` must reference a registered device (the read handler does not enforce device existence), so a

typo'd or non-existent device id can be added to a group; impact is low (it simply never matches a real device) but it pollutes cohort data.

4.13.4 Staged-rollout abuse (create / evaluate; HALT-wins as a safety control)

- **STRIDE:** Denial of service, Tampering, Elevation of privilege.
- **Attack:** An actor with rollout rights (operator/admin) — or an attacker who reaches those rights via §4.8 — creates a malicious/garbage rollout plan, or feeds a forged health verdict to `POST .../rollout/evaluate` to **force-advance a bad rollout** (suppress a halt) or **force-halt a good one** (DoS the update channel). Because `evaluate` accepts a client-supplied `RolloutVerdict` (`success_rate`, `error_rate`, `post_boot_health_failed`), the verdict is an attacker-influencable input to the gate.
- **Impact:** A bad release advanced fleet-wide (chains into §4.1/§4.3 impact); or a good rollout stuck/halted (denial of update, §4.11); or wasted rollout state.
- **Mitigation in our design (as implemented):**
 - **Authorization.** Create + evaluate are **operator/admin** (`requireRole(RoleOperator, RoleAdmin)`); read is **viewer+**. The device role cannot create or evaluate a rollout. Both mutations are **audited** on 2xx (§4.13.1).
 - **Plan validation in the brick.** `handleCreateRollout` delegates to the `ota-rollout-engine` brick (`s.rollout.CreateAndStart`); the handler doc records the brick validates the phase plan (strictly-increasing percentages ending at 100, thresholds in $[0,1]$) and a plan violation maps to **400** — so a garbage plan is rejected at creation, not silently started.
 - **Existence checks.** `handleCreateRollout` first loads the deployment (404 if absent); `handleEvaluateRollout` maps `engine.ErrNotFound` to 404 — so rollouts cannot be created/driven for non-existent deployments.
 - **HALT-wins as a safety control.** The `evaluate` path returns the engine decision (advance / hold / halt / complete). The engine's halt-on-breach is the **fail-safe** default: a verdict that breaches the error/health threshold halts the rollout rather than advancing. This is the safety-relevant property — the *only* way an attacker-forged verdict causes fleet-wide harm is by **suppressing** a halt (reporting a healthy verdict for an unhealthy cohort), not by injecting one; a forged *unhealthy* verdict can only halt (deny), which is the safe direction. (Cross-ref §4.10 telemetry spoofing — the verdict that *should* feed `evaluate` is telemetry-derived; in 1.0.1+ the `evaluate` input must be bound to aggregated, device-identity -gated telemetry rather than a free client body, or the gate is only as trustworthy as whoever can call `evaluate`.)
- **Residual risk: Moderate.** The dominant residual is that **evaluate trusts a client-supplied verdict body** — it is not, in the read code, computed server-side from stored telemetry. An operator (or §4.8 attacker with operator rights) can therefore submit a fabricated healthy verdict to suppress a halt. HALT-wins bounds the *unhealthy* direction (forced halt = DoS only), but the *suppress-halt* direction is not mitigated at this layer and depends entirely on who can authenticate as operator/admin. **UNVERIFIED:** whether the brick or a future wiring computes the verdict from repo telemetry (§4.10) rather than the request body; and the exact halt-precedence semantics (“HALT-wins”) are asserted from the handler's documented decision set, not from reading the brick's evaluator — carried UNVERIFIED against the brick source.

4.13.5 Recall / rollback authorization + anti-downgrade interplay

- **STRIDE:** Tampering, Denial of service, Repudiation.
- **Attack:** An actor with recall rights (operator/admin) — or a §4.8 attacker who reaches them — issues `POST /deployments/{id}/recall` to roll a deployment's release back to a chosen `to_release_id`. The security tension is that a **recall to N-1 is, by intent, a downgrade** of the deployed release — the same *shape* of action that the **anti-downgrade guarantee** (server-side version monotonicity at upload + on-device AVB rollback-index, §4.3) exists to *prevent* when an attacker mounts it (§4.3 rollback/downgrade attack). The model must distinguish “operator-gated legitimate recall” from “attacker downgrade”.
- **Impact:** If recall could silently override anti-downgrade, an attacker who obtains operator rights would have a *sanctioned* path to push the fleet back to vulnerable firmware — collapsing §4.3's protection. Conversely, if anti-downgrade is absolute, a legitimate emergency recall to a known-good N-1 may be blocked on-device.
- **Mitigation in our design (as implemented):**
 - **Authorization + audit.** Recall is **operator/admin** (`requireRole(RoleOperator, RoleAdmin)`); the device role cannot trigger a recall. The action is **audited** on 2xx (`DEPLOYMENT_*` verb via `auditMiddleware`), and additionally `handleRecall` records a dedicated **append-only rollback_history row** (`AppendRollback`, `INSERT-only`, `kind=rollback`, `from_release_id`=the deployment's current release, `to_release_id`=requested, `triggered_by`=`claims.Subject`, optional reason). So a recall is non-repudiable and attributable on two independent append-only trails.
 - **Validation gates.** `handleRecall` requires the deployment to exist (404), `to_release_id` non-empty (400), the deployment to have a current release to roll back *from* (400), and the **target release to exist** (404) — so a recall cannot point at a non-existent release.
 - **Recall is the audited control + record, not (yet) the re-deploy.** The handler's own doc is explicit: “The actual N-1 re-deployment is the deployment engine's job (tracked separately); this endpoint is the audited control + record.” If an active rollout exists it is intended to be marked rolled-back via an abort evaluation. So at this layer recall **records intent**; it does not itself bypass any device-side check.
 - **The anti-downgrade invariant is bootloader-enforced on-device (§4.3 layer 2).** The AVB rollback-index check is enforced by the bootloader, which the Helix agent and control plane cannot weaken. A recall to N-1 that shares the *same* rollback index as N would be permitted by the bootloader (rollback indexes bump only at security milestones); a recall to an N-1 with a *lower* rollback index would be **rejected by the bootloader** regardless of operator intent.
- **Residual risk: Moderate, and the recall-vs-anti-downgrade tension is explicitly an open decision.**
 - **The interplay is currently a documented seam, not a resolved mechanism.** Because the implemented `handleRecall` only records the rollback intent and defers the actual re-deploy to a separate (not-yet-read) deployment-engine path, **how a recall to a lower-rollback-index N-1 actually reconciles with the bootloader's anti-downgrade is UNVERIFIED.** Two outcomes are possible and the code read does not decide between them: (a) the recall is honored at the server (record + re-deploy) but the device's bootloader still **refuses** to boot an image below the stored rollback index — so the recall silently fails on-device for security-

milestone downgrades; or (b) a future mechanism deliberately allows the downgrade (e.g. signing an N-1 image that carries an acceptable rollback index, or an operator-gated rollback-index exception). **Per §11.4.6 this is not guessed** — it is flagged for an operator/architecture decision (Continuation item 5).

- **Recall is operator-gated by design, which is the intended distinguisher** from the §4.3 attacker downgrade: a §4.3 attacker without operator rights and without the signing key cannot forge a recall, and even an attacker *with* operator rights produces an attributable, append-only `rollback_history` + audit record. The recall path therefore does not *add* a new unauthenticated downgrade vector beyond §4.8 (whoever can authenticate as operator). The open risk is the *legitimate-but-abused* operator and the unresolved on-device reconciliation, not a new anonymous attack surface.
- **The literal label “G1” for the anti-downgrade guarantee is UNVERIFIED** against the design corpus: the master design (§ “Hard guarantees”, line ~121) and §4.3 here name it “version monotonicity” / “anti-downgrade”, not “G1”. The *mechanism* is verified; the *label* is not.

5. Residual-risk summary

#	Threat	MVP residual	Fully closed by
4.1	Forged / malicious artifact upload	Low–moderate (key-dependent)	dual-gate verify (MVP) + TUF targets/thresholds (1.0.1+)
4.2	Signature / key compromise	High (accepted)	TUF thresholds + offline keys (1.0.1+)
4.3	Rollback / downgrade	Moderate	AVB rollback-index (MVP, board-conditional) + TUF freshness (1.0.1+)
4.4	Mix-and-match	Moderate	atomic payload + AVB (MVP) + TUF snapshot binding (1.0.1+)
4.5	Endless-data	Moderate	length+hash verify-before-apply (MVP) + TUF authenticated length (1.0.1+)
4.6	MITM on poll / download	Low (integrity) / moderate (confidentiality + fake-availability)	TLS 1.3 + device re-verify (MVP); pinning/mTLS + signed metadata (1.0.1+)
4.7	Device impersonation	Moderate	KeyStore-bound token (MVP); mTLS + Director

#	Threat	MVP residual	Fully closed by
4.8	Dashboard auth bypass	Moderate (impl-dependent)	per-device targeting (1.0.1+) OAuth2/JWT/RBAC + audit (MVP); spec-level role matrix/MFA
4.9	Build-pipeline supply-chain	High (accepted)	TUF/Uptane compromise resilience + dual-repo (1.0.1+)
4.10	Telemetry spoofing	Moderate	TLS + device identity + threshold aggregation (MVP); per-cohort gate re-eval (1.0.1+)
4.11	Denial of update	Moderate	rate-limit/jitter + safe A/B degrade (MVP); TUF anti-freeze/anti-mirror (1.0.1+)
4.12	Slot corruption	Low (board-conditional)	AVB/dm-verity/update_verifier/auto-rollback (MVP, AOSP-owned)
4.13.1	Audit-log integrity + reads/failed-mutations not audited	Moderate	append-only store + admin-only read + ordered middleware (MVP); DB-grant revoke + separate security-event log (open)
4.13.2	Telemetry IDOR (own-device-only)	Low (token-conditional)	handler subject check + role gate (MVP); hardened device identity / mTLS (§4.7, 1.0.1+)
4.13.3	Device-group membership authorization	Moderate	operator/admin writes + admin-only delete + audit (MVP); per-group ACL / scoping (future)
4.13.4	Staged-rollout abuse	Moderate	operator/admin + brick plan-validation +

#	Threat	MVP residual	Fully closed by
4.13.5	Recall vs anti-downgrade tension	Moderate (open decision)	HALT-wins fail-safe (MVP); server-computed verdict from telemetry (1.0.1+, §4.10) operator-gated + dual append-only trails + bootloader rollback-index (MVP); on-device recall reconciliation = operator decision

Headline. The two **High, explicitly accepted** residuals — **signing-key compromise (4.2)** and **build-pipeline supply-chain (4.9)** — are the same single-signing-key exposure that ADR-0002 §5.2 documents and that motivates the phased TUF/Uptane adoption. The on-device **zero-corruption** posture (4.12, and the safe-degrade in 4.3/4.11) is **strong but conditional on the board actually shipping locked AVB** — an UNVERIFIED item that must close.

6. Compliance notes (HelixConstitution)

Clause numbers/labels follow the corpus convention; the authoritative HelixConstitution text is not present in this repository, so clause wording is **UNVERIFIED** against the source (consistent with `documentation_standards.md` §8 and ADR-0002 §7).

Clause	Label (per corpus)	How this document complies
§11.4.61	Table of contents mandatory	Metadata table first, ToC immediately after (this doc top).
§7.1	No-bluff / evidence-only	Every mitigation cites a master/ADR/AVB-note section; no capability is claimed that the MVP does not ship.
§11.4.6	No-guessing	Unconfirmed facts (board AVB lock, rollback-index backend, AIDL surface, RBAC details, brick fit) carried as UNVERIFIED , never invented.
§11.4.123	Rock-solid-proof	Unmappable-until-Constitution-present (per ADR-0002 §7): the

Clause	Label (per corpus)	How this document complies
		clause text is not in-repo, so a definitive mapping is out of scope; provisionally, every High/board-conditional residual is tied to a named, scheduled closure (1.0.1+ or AVB-note Continuation) rather than asserted as resolved. UNVERIFIED against clause text. Only verified catalogue submodules referenced (auth, security, database, Storage, observability, eventbus, ratelimiter, middleware, http3, recovery, Herald, config, cache; Auth-KMP, Security-KMP, Storage-KMP, Config-KMP; dashboard React bricks); none invented. Threats mapped to the decoupled units/ boundaries (validator, rollout engine, protocol, update-engine bridge, telemetry schema) per master §4/§10. Safety-critical signing-verify/apply/rollout-gate paths target ≥90% under the four-layer + mutation regime (master §13), referenced in §4.1, §4.12. This document is subject to the mandatory adversarial code-review subagent before acceptance (master §14).
§11.4.74	Catalogue-first reuse	
§11.4.28	Decoupling	
§1 / §1.1	Four-layer testing + mutation	
§11.4.125	Code-review gate	

7. Open / UNVERIFIED items

1. **Board AVB conformance** — whether the RK3588 / Orange Pi 5 Max build ships a conformant `boot_control` HAL + **locked AVB**; the `slot-retry-`

count default; the rollback-index storage backend (RPMB vs persistent partition vs TEE); the Android 15 IBootControl AIDL surface.

UNVERIFIED. Gates the firmness of §4.3 and §4.12 residuals. (android-avb-rollback.md §Issues, §12; ADR-0002 §8 item 8.)

2. **Catalogue-brick fit for the signing/verify seam** — whether security / Security-KMP host the verify (and future TUF-role) primitives without bespoke crypto. **UNVERIFIED.** (ADR-0002 §8 item 9.) Affects §4.1, §4.2.
3. **Dashboard authorization specifics** — concrete RBAC role matrix, token lifetime/refresh, per-endpoint authorization, audit coverage of all state-changing calls, MFA. **UNVERIFIED** (component-spec level). Affects §4.8.
4. **Resource-bound specifics** — agent download-buffer / decompression caps, intake size caps, and control-plane rate-limit thresholds / capacity sizing. **UNVERIFIED** (component-spec / deployment level). Affects §4.5, §4.11.
5. **mTLS / certificate pinning decision** — recorded only as “evaluated” in the MVP (master §6); not committed. Affects §4.6, §4.7.
6. **HelixConstitution clause text** — not in-repo; all §11.4.x citations **UNVERIFIED** against the source (documentation_standards §8; ADR-0002 §7).
7. **Deferred TUF/Uptane controls** — rollback-freshness, snapshot/target binding, authenticated length, anti-freeze/anti-mirror, threshold + offline-key compromise resilience, and per-device Director targeting are **scheduled for 1.0.1+** (ADR-0002 §4.3) and are **not** MVP mitigations; this document treats them as future closures, not current controls.
8. **Audit-log DB-grant enforcement** — append-only is enforced at the store.Repository interface (no Update/Delete) + INSERT-only SQL; **UNVERIFIED** whether the deployment revokes UPDATE/DELETE on helix_ota.audit_logs / rollback_history from the app DB role. Affects §4.13.1.
9. **Security-event vs audit log** — reads and failed mutations are intentionally **not** audited (handler doc); whether a separate security-event log (auth failures, RBAC denials, repeated 4xx) is warranted is an open decision. Affects §4.13.1.
10. **Rollout evaluate verdict source** — POST .../rollout/evaluate accepts a client-supplied RolloutVerdict; **UNVERIFIED** whether the verdict is (or will be) computed server-side from stored telemetry instead of trusting the request body, and the exact HALT-precedence semantics in the ota-rollout-engine brick (asserted from handler doc, not read from brick source). Affects §4.13.4 (cross-ref §4.10).
11. **Recall  anti-downgrade reconciliation** — handleRecall records intent + an append-only rollback_history row but defers the actual N-1 re-deploy to a separate deployment-engine path not read here; **how a recall to a lower-rollback-index N-1 reconciles with the bootloader-enforced AVB anti-downgrade (§4.3) is UNVERIFIED and needs an operator/architecture decision** (honor-but-device-refuses vs deliberate operator-gated downgrade mechanism). Affects §4.13.5. **RESOLVED (2026-06-08, operator): honor AVB — recall is FORWARD-FIX ONLY.** handleRecall now supersedes the current deployment + creates a NEW active deployment of the target release; the update-check anti-downgrade invariant ensures no device is offered a version $\leq$ its current, so the bootloader-enforced AVB anti-rollback is honored by construction. A true sub-rollback-index downgrade is NOT offered. No longer an open decision.
12. **“G1” label** — the prompt-supplied label “G1 anti-downgrade invariant” is **UNVERIFIED** against the design corpus, which names the guarantee “version monotonicity” / “anti-downgrade”; the mechanism is verified, the literal label is not. Affects §4.13.5.

13. **Group member device-existence + per-group scoping** — `member-add` validates non-empty `device_id` + group existence but **UNVERIFIED** whether it requires a *registered* device; authorization is role-coarse (any operator edits any group), no per-group ACL. Affects §4.13.3.

8. Sources

All paths relative to `docs/research/main_specs/`:

- [`00-master/2026-06-07-helix-ota-design.md`](#) — §1 (vision, hard guarantees, non-goals), §2 (locked decisions), §3 (stack), §4 (architecture, decoupling), §5 (MVP definition), §6 (security & trust model), §7 (data model), §8 (rollout engine), §9 (telemetry), §10 (submodule reuse + new repos), §13 (testing), §14 (execution / code-review).
- [`research/adr/adr-0002-supply-chain-trust.md`](#) — §1 (why plain signing is not enough), §2 (decision drivers), §3.1–§3.2 (plain-signing gap; attack classes TUF closes), §4.1 (MVP trust model), §4.2 (MVP-forward signer interface), §4.3 (1.0.1+ sequencing), §5.1–§5.2 (consequences), §7 (compliance notes), §8 (open/UNVERIFIED).
- [`research/stacks/android-avb-rollback.md`](#) — §2 (executive summary), §3 (AVB chain of trust), §4 (dm-verity + FEC), §5 (`boot_control` HAL), §6 (`update_verifier`), §7 (automatic rollback), §8 (rollback-index / anti-downgrade), §9 (zero-corruption guarantee), §10 (agent MUST/MUST-NOT), §12 (open/UNVERIFIED), Issues + Continuation (board specifics).
- [`00-master/documentation\_standards.md`](#) — §2 (metadata table), §3 (ToC requirement, §11.4.61), §8 (anti-bluff/UNVERIFIED), §9 (canonical submodule catalogue).

§4.13 — implemented handlers (read directly, not via design docs). All paths relative to the repository root:

- `server/internal/api/server.go` — protected route group + per-route `requireRole(...)` RBAC (lines 122–168), middleware ordering `authMiddleware()` → `auditMiddleware()`, public auth endpoints. Source of the §4.13 RBAC table.
- `server/internal/api/handlers_audit.go` — `auditMiddleware` (runs after handler + RBAC; mutating
 - 2xx only; reads/failed mutations not audited; best-effort write), `deriveAuditAction` (route-template not raw path; members → `group_member`), `auditResourceID`, `handleListAudit` (admin-only). §4.13.1.
- `server/internal/api/handlers_telemetry.go` — `handleDeviceTelemetry` object-level subject check (! `isPrivileged(claims) && claims.Subject != deviceID` → 403), `handleTelemetryOverview` (aggregate-only). §4.13.2.
- `server/internal/api/handlers_group.go` — group + member CRUD handlers. §4.13.3.
- `server/internal/api/handlers_rollout.go` — `handleCreateRollout` (brick plan-validation → 400), `handleGetRollout`, `handleEvaluateRollout` (client-supplied `RolloutVerdict`; decision set advance/hold/halt/complete). §4.13.4.

- `server/internal/api/handlers_recall.go` — `handleRecall` (records intent + append-only `rollback_history`; defers N-1 re-deploy), `handleListRollbacks`. §4.13.5.
- `server/internal/api/handlers_device.go` — `isPrivileged` (admin OR operator OR viewer; device never privileged), device-status subject check. §4.13.2.
- `server/internal/api/middleware.go` — `requireRole` (any-allowed-role), `claimsFrom`. §4.13 RBAC.
- `server/internal/api/token.go` — role constants (admin/operator/viewer/device). §4.13 RBAC.
- `server/internal/store/store.go` — Repository interface exposing only `AppendAudit/ListAudit`
 - `AppendRollback/ListRollbacks` (no `Update/Delete` → append-only). §4.13.1, §4.13.5.
- `server/internal/store/postgres.go` + `memory.go` — INSERT-only audit/rollback impls with monotonic seq ordering. §4.13.1, §4.13.5.